

TestPlatform

Rapport d'Execution de Campagne

Campagne #19

Projet: Projet #17

Version: N/A

Environnement: Environnement #12

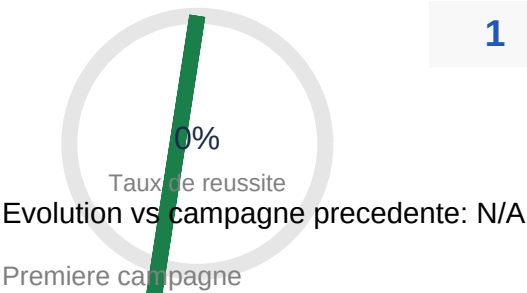
Date: 03/06/2026 10:02

ECHEC

Resume executif

Taux de reussite

Total	Succes	Echecs	Erreurs	Duree
1	0	1	0	211.3 s



Informations du contexte

Projet	Projet #17
Description	N/A

Suites

Nom	Type	Depot Git	Branche
Suite UNIT	UNIT	N/A	main

Cas de test

Titre	Type	Priorite	Risque	Script
Test #32	UNIT	N/A	N/A	IA

Environnement

URL Web	N/A
URL API	N/A
Repository	https://github.com/ilyeschaabani/Automatisation_intelligente_des_tests_applicatifs
Branche	main
Base de donnees	POSTGRESQL

Resultats details

Test #32

FAILURE

Type	UNIT	Duree	9.2 s
Script	IA	Erreur	Maven exit code : 1

Analyse IA

Test #32

Logs (extrait)	<pre>R] [ERROR] For more information about the errors and possible solutions, please read the following articles: [ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException</pre>
Analyse IA	La cause probable de l'échec est probablement liée au manque d'initialisation des mocks dans le setUp() de votre test. Lorsqu'aucun mock n'est initialisé, tous les appels à ces objets deviennent réalistes et le comportement du système ne sera pas contrôlé par les tests (par exemple, l'appel au repository pour enregistrer un nouveau cas de test n'est plus faux). Pour corriger cela, vous devriez appeler Mockito.initMocks(this) dans le setUp() avant d'ouvrir les mocks. Cela va initialiser tous les champs annotés avec @Mock sur l'objet de test : Correction: <pre>```java @BeforeMethod public void setUp() { MockitoAnnotations.initMocks(this); } ```</pre> De plus, la réponse ci-dessous est valide uniquement si l'erreur est liée à l'absence de configuration initiale des mocks dans le setUp(). À partir du code que vous avez fourni et en raison du message d'erreur non reconnu, il semble qu'il n'y ait pas eu d'échec sur la phase de compilation. En termes de meilleures pratiques de test, une suggestion pour améliorer votre code est de tester le comportement normal et l'exception en dehors des mocks. Vous pouvez utiliser Mockito avec le framework TestNG pour simuler les dépendances et vérifier le comportement standard et exceptionnel du système, ce qui permet d'obtenir une couverture de code complète : <pre>```java @Test(expectedExceptions = RuntimeException.class) public void testSaveWithNullCase() { when(testCaseRepository.save((TestCase) any())).thenReturn(null); TestCase tc = new TestCase(); assertTrue(testCaseService.save(tc)); } ```</pre> Cela permet d'obtenir une couverture de code complète et une meilleure prédiction du comportement de votre application dans des scénarios réels.

Evaluation UX

Aucun test UX execute pour cette campagne

Metriques

Top 5 des tests les plus lents

Test	Suite	Duree
Test #32	Suite UNIT	9.2 s

Temps d'execution par type

Type	Duree moyenne
UNIT	9.2 s

Tests instables

Aucun test instable detecte

Recommandations

- Test #32: La cause probable de l'échec est probablement liée au manque d'initialisation des mocks dans le setUp() de votre test. Lorsqu'aucun mock n'est initialisé, tous les appels à ces objets deviennent réali...

Annexe - Scripts generes par IA

Test #32

```
package suites.unit;

import com.pfe.platform.ms_gestion.dto.request.CreateTestCaseRequest;
import com.pfe.platform.ms_gestion.dto.response.TestCaseResponse;
import com.pfe.platform.ms_gestion.entity.Project;
import com.pfe.platform.ms_gestion.entity.ProjectMember;
import com.pfe.platform.ms_gestion.entity.TestCase;
import com.pfe.platform.ms_gestion.entity.TestSuite;
import com.pfe.platform.ms_gestion.repository.ProjectMemberRepository;
import com.pfe.platform.ms_gestion.repository.TestCaseRepository;
import com.pfe.platform.ms_gestion.repository.TestSuiteRepository;
import com.pfe.platform.ms_gestion.security.SecurityUtils;
import com.pfe.platform.ms_gestion.service.LlmService;
import com.pfe.platform.ms_gestion.service.SkeletonExtractorService;
import com.pfe.platform.ms_gestion.service.TestCaseService;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockedStatic;
import org.mockito.Mockito;
import org.mockito.MockitoAnnotations;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;
import java.util.Optional;

public class TestCaseServiceTest {

    @Mock private TestCaseRepository testCaseRepository;
    @Mock private TestSuiteRepository testSuiteRepository;
    @Mock private ProjectMemberRepository projectMemberRepository;
    @Mock private LlmService llmService;
    @Mock private SkeletonExtractorService skeletonExtractorService;
    @InjectMocks private TestCaseService testCaseService;

    private AutoCloseable mocks;
    private MockedStatic<SecurityUtils> mockedSecurity;

    @BeforeMethod
    public void setUp() {
        mocks = MockitoAnnotations.openMocks(this);
        mockedSecurity = Mockito.mockStatic(SecurityUtils.class);
        mockedSecurity.when(SecurityUtils::getCurrentUserId).thenReturn(1L);
    }

    @AfterMethod
    public void tearDown() throws Exception {
        mockedSecurity.close();
        mocks.close();
    }

    @Test
    public void testAddTestCase() {
        // Arrange
        Long suiteId = 1L;

        Project project = new Project();
        project.setId(10L);
        project.setName("Test Project");

        ProjectMember member = new ProjectMember();
        member.setRole(ProjectMember.Role.ADMIN);
        member.setUserId(1L);
```

```
member.setProject(project);

TestSuite suite = new TestSuite();
suite.setId(suiteId);
suite.setType(TestSuite.TestType.UNIT);
suite.setProject(project);

CreateTestCaseRequest request = new CreateTestCaseRequest();
request.setTitle("Login test");
request.setType("UNIT");
request.setRiskLevel("LOW");
request.setPriority(1);
request.setDescriptionAI("Test the add method");

Mockito.when(testSuiteRepository.findById(suiteId))
    .thenReturn(Optional.of(suite));
Mockito.when(projectMemberRepository.findByIdAndUserId(
    Mockito.any(), Mockito.eq(1L)))
    .thenReturn(Optional.of(member));
Mockito.when(llmService.generateTestCode(
    Mockito.any(), Mockito.any(), Mockito.any(), Mockito.any()))
    .thenReturn("public class FakeTest {}");
Mockito.when(testCaseRepository.save(Mockito.any()))
    .thenAnswer(inv -> {
        TestCase tc = inv.getArgument(0);
        tc.setSuite(suite);
        return tc;
    });

// Act
TestCaseResponse response = testCaseService.add(suiteId, request);

// Assert
Assert.assertNotNull(response);
Assert.assertEquals(response.getTitle(), "Login test");
Assert.assertEquals(response.getRiskLevel(), "LOW");
Mockito.verify(testCaseRepository, Mockito.times(1)).save(Mockito.any());
Mockito.verify(llmService, Mockito.times(1))
    .generateTestCode(Mockito.any(), Mockito.any(), Mockito.any(), Mockito.any());
}
```